

Building a Mini GPT from Scratch: A Character-Level Transformer

Naga Sai Haritha Gottumukkala
Department of Applied Data Science
San Jose State University

DATA 266 – Generative AI and Large Language Models
SID4: 1598 SEED: 1598

Abstract—In this homework, I built a small GPT model from scratch using PyTorch. My main goal was to understand what actually happens inside a GPT model instead of using a ready-made Transformer.

I used the provided Shakespeare text and treated every character as a token. The file had 1,738 characters and 49 unique characters. Using a sequence length of 128 gave me 1,610 input-target pairs for training.

The model has four attention heads and two decoder blocks. I manually implemented the masked self-attention instead of using `nn.MultiheadAttention` or `nn.Transformer`. The final model had 425,777 trainable parameters.

I trained the model for eight epochs using Adam with a learning rate of 0.0003 and batch size 32. The training loss came down from 3.1064 in the first epoch to 1.8965 in the last epoch.

After training, I generated text starting with the prompt `ROMEO:`. I compared greedy decoding, three temperature settings, and two top- k settings. From my outputs, top- k with $k = 2$ gave slightly more readable short phrases, while temperature 1.5 gave the most varied output. The generated text was still far from perfect, mainly because the training text was very small.

Index Terms—GPT, Transformer, PyTorch, self-attention, causal mask, character-level model, temperature sampling, top- k sampling

I. INTRODUCTION

The purpose of this homework was to build a small GPT model and understand its main parts by implementing them myself.

Normally, when we use a language model, we only see the input and the generated answer. In this assignment, I worked with what happens between those two steps. I had to convert text into tokens, create embeddings, add position information, calculate attention, hide future characters using a causal mask, train the model, and finally generate new text.

The model used in this homework is a decoder-only Transformer. Its job is simple: look at the characters that have already appeared and predict what character should come next.

I used character-level tokenization instead of word-level or subword tokenization. This means that the model predicts one character at a time. For example, after seeing a sequence ending with “Rom”, the model is trained to predict what character is likely to follow.

Another important part of the homework was that I could not use `nn.Transformer`, `nn.MultiheadAttention`, or a pretrained Hugging Face Transformer. I therefore implemented

the attention calculation manually using normal PyTorch operations.

After training, I also compared different ways of choosing the next character. This helped me see that the same model can generate very different text depending on the decoding method.

II. SETUP AND REPRODUCIBILITY

The personal values required for the course are shown in Table I. Only SEED was directly needed for this homework, but I kept all the values in my notebook as required.

TABLE I
PERSONAL PARAMETERS

Parameter	Value
SID4	1598
SEED	1598
SLICE	598
HP_ID	2
CLS_A	8
CLS_B	5

I used the same seed, 1598, for Python, NumPy, and PyTorch. I also used the same seed while shuffling the training examples.

My final run used Python 3.13.9 and PyTorch 2.14.0 on CPU.

I kept the seed fixed because I wanted my final outputs to be repeatable as much as possible. I also reset the same seed before the different sampling experiments.

III. DATA PREPARATION

A. Character-Level Tokenization

I started with the provided `shakespeare.txt` file.

Instead of splitting the text into words, I treated each unique character as one token. Characters included letters, spaces, punctuation, and line breaks.

The dataset had:

- 1,738 total characters,
- 49 unique characters.

I created two mappings:

$$\text{char_to_idx} \quad (1)$$

to convert each character into an integer, and

`idx_to_char` (2)

to convert an integer back into a character.

I also encoded the full text and decoded it again to check that the original text was recovered correctly.

TABLE II
DATASET SUMMARY

Item	Value
Dataset	Shakespeare text
Total characters	1,738
Vocabulary size	49
Sequence length	128
Training pairs	1,610

B. Sliding Window

The required sequence length was 128.

For every training example, I took 128 characters as the input. The target contains the same sequence shifted by one position.

In a very small example:

Input: ABCD

Target: BCDE

The actual homework follows the same idea using 128 characters.

Mathematically,

$$x = [t, t + 1, \dots, t + 127] \quad (3)$$

and

$$y = [t + 1, t + 2, \dots, t + 128]. \quad (4)$$

This gave me 1,610 training pairs.

I printed two decoded examples in the notebook so that I could visually confirm that the target was shifted correctly.

One of the examples started like this:

```
Input:
ROMEO:
But, soft! what light through yonder window breaks?

Target:
OMEO:
But, soft! what light through yonder window breaks?
```

Seeing the examples made it easier for me to confirm that the dataset was doing next-character prediction correctly.

IV. BUILDING THE MINI GPT

A. Token and Position Embeddings

After tokenization, every character is only an integer.

For example, a character may be represented as 5 or 20, but the number itself does not describe its meaning. I therefore used a token embedding layer to convert every character ID into a learnable 128-dimensional vector.

I also used positional embeddings.

This is needed because the model should know where a character appears inside the sequence. The same character appearing near the beginning and near the end of a sentence can have a different role.

The two embeddings are added together:

$$X = E_{\text{token}} + E_{\text{position}}. \quad (5)$$

I think of the token embedding as telling the model “what character is this?”, while the positional embedding tells it “where is this character?”.

B. Multi-Head Self-Attention

I manually implemented the attention layer.

The model first creates Query, Key, and Value representations:

$$[Q, K, V] = XW_{QKV}. \quad (6)$$

The hidden size is 128 and I used four attention heads. Therefore, each head works with

$$\frac{128}{4} = 32 \quad (7)$$

features.

The attention score is calculated using

$$S = \frac{QK^T}{\sqrt{d_h}}. \quad (8)$$

Here, d_h is the size of one attention head.

In simple terms, the attention score helps the model decide which earlier characters are useful when processing the current character.

C. Causal Mask

The causal mask was one of the most important parts of this model.

When predicting the next character, the model should not be allowed to see characters that come later in the sequence. If it could see them, it would already know the answer during training.

I created an upper-triangular mask and replaced future attention scores with negative infinity before applying softmax.

The idea is:

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (9)$$

So each position can look at itself and earlier positions, but not future positions.

I also tested the mask in my notebook. I changed only the second half of an input sequence and checked the first-half predictions. The earlier predictions stayed the same, while the later predictions changed.

This gave me a direct check that future characters were not leaking into earlier predictions.

D. Decoder Block

Each decoder block contains:

- masked multi-head attention,
- LayerNorm,
- residual connections,
- a feedforward network.

My feedforward network is:

$$128 \rightarrow 512 \rightarrow 128 \quad (10)$$

using

$$\text{Linear} \rightarrow \text{GELU} \rightarrow \text{Linear}. \quad (11)$$

I used the pre-LayerNorm structure from the demo:

$$X_1 = X + \text{Attention}(\text{LayerNorm}(X)), \quad (12)$$

and then

$$X_2 = X_1 + \text{FFN}(\text{LayerNorm}(X_1)). \quad (13)$$

The residual connection means that the original input is added back after each main operation. This helps preserve information while the model learns new transformations.

E. Final Model

I stacked two decoder blocks.

After the decoder blocks, I used a final LayerNorm followed by a linear layer that produces one score for every character in the vocabulary.

My final model settings are shown in Table III.

Setting	Value
Vocabulary size	49
Sequence length	128
Hidden dimension	128
Attention heads	4
Features per head	32
Decoder blocks	2
Feedforward size	512
Parameters	425,777

The final model had 425,777 trainable parameters.

V. TRAINING THE MODEL

I trained the model using Adam.

The settings I used were:

TABLE IV
TRAINING SETTINGS

Setting	Value
Optimizer	Adam
Learning rate	0.0003
Batch size	32
Epochs	8
Loss	Cross-entropy
Device	CPU

Cross-entropy loss compares the character predicted by the model with the actual next character.

I recorded the average loss after every epoch.

TABLE V
TRAINING LOSS

Epoch	Loss
1	3.106401
2	2.479756
3	2.278498
4	2.175339
5	2.104990
6	2.040887
7	1.971539
8	1.896454

The loss decreased in every epoch.

It started at about 3.1064 and ended at about 1.8965.



Fig. 1. Training loss of the Mini GPT over eight epochs.

The largest improvement happened during the first few epochs. After that, the loss continued to decrease, but more slowly.

From this graph, I can say that the model was getting better at the training next-character prediction task.

However, I only measured training loss in this homework. I did not use a separate validation set, so this graph does not tell me how well the model would perform on completely unseen text.

VI. GENERATING TEXT

After training, I used the prompt

ROMEO:

for every decoding method.

I generated 300 new characters for each experiment.

A. Greedy Decoding

Greedy decoding always picks the character with the highest model score.

It is simple and predictable, but in my experiment it became very repetitive.

For example:

```
ROMEO:
I thy thame noull I thame,
thame the the the thais:
...
The the ses
...
```

I noticed repeated patterns like “the the the” and repeated groups of letters.

So even though greedy decoding always chooses the model’s favorite next character, that did not mean the final passage was the easiest to read.

B. Temperature Sampling

Temperature changes how strongly the model prefers its highest-scoring characters.

I tested:

$$T = 0.5, \quad T = 1.0, \quad T = 1.5. \quad (14)$$

For temperature 0.5, the model favored the more likely characters.

My output started with:

```
ROMEO:
What name thame
Why name name
I amend of whe
...
```

This was somewhat controlled, but repeated letters and broken words still appeared later.

At temperature 1.0, I got more variation, but many of the phrases were still difficult to understand.

At temperature 1.5, the result became much more random:

```
ROMEO:
Weat befucos ngall?
It Mw ik ULI grn nof latcwhouevel
Why;Lit; d, amul my dyos, am,.
...
```

This output had more unusual capitalization, punctuation, and character combinations.

It looked more varied than the other temperature outputs, but it was also harder to read.

C. Top- k Sampling

For top- k sampling, I only allowed the model to sample from its k most likely next characters.

I tested:

$$k = 2 \quad (15)$$

and

$$k = 20. \quad (16)$$

With $k = 2$, the model had very few choices.

One part of the output was:

```
ROMEO:
I thame
I nome that thou name nore no whe the the
...
```

I could recognize more short fragments here, although repetition was still a problem.

With $k = 20$, the model had many more possible characters to choose from. The output became more varied, but I also saw more malformed words.

VII. WHAT I OBSERVED FROM THE DECODING METHODS

Table VI summarizes what I noticed from my final outputs.

A. Most Coherent Output

From my final outputs, I found top- k with $k = 2$ slightly easier to read.

It produced some recognizable short fragments and did not become as random as the high-temperature output.

I would not call the full passage fluent or completely meaningful. None of my generated passages reached that level.

So when I say that $k = 2$ was the most coherent, I mean that it was the most readable relative to the other samples I generated.

B. Most Diverse Output

Temperature 1.5 was clearly the most varied output in my experiment.

It generated more unusual characters, punctuation, capitalization, and word-like combinations.

The disadvantage was that much of the text stopped looking meaningful.

This helped me understand that more diversity is not always better. There is a trade-off between trying different possibilities and keeping the generated text readable.

VIII. FAILURE AND ERROR ANALYSIS

The model trained successfully, but the generated text still had several problems.

A. Repeated Text

The first issue I noticed was repetition.

Greedy decoding repeatedly produced words such as “the”, while top- k with $k = 2$ repeatedly produced words such as “name”.

This showed me that when the model has only a few strong choices, it can easily fall into the same pattern again and again.

TABLE VI
MY COMPARISON OF THE DECODING METHODS

Method	Setting	What I Saw	My Interpretation
Greedy	Highest score	Repeated phrases such as “the the the” and repeated letters.	The output was very restricted and got stuck in repeated patterns.
Temperature	0.5	Started with readable-looking text such as “What name,” but later became repetitive.	Lower temperature made the choices safer, but it did not completely solve repetition.
Temperature	1.0	The output had more variation, but many words were unclear.	This gave a balance between safer and more random choices.
Temperature	1.5	There was much more unusual spelling, capitalization, and punctuation.	This was the most varied output, but it was also harder to read.
Top- k	2	Produced short recognizable fragments such as “that thou name.” It also repeated common words.	This was slightly easier for me to read than the other outputs.
Top- k	20	More varied than $k = 2$, but many generated words were malformed.	Giving the model more choices increased variety but reduced control.

B. Broken Words

Another common problem was incomplete or made-up words.

This happened especially with temperature 1.5 and top- k with $k = 20$.

Those settings allow more possible characters, so the output becomes more creative, but it can also move away from realistic words.

C. Very Small Dataset

The Shakespeare file used for this homework contains only 1,738 characters.

Even though the sliding-window process created 1,610 training pairs, many of those pairs share most of the same characters because the window moves only one position at a time.

So the model is still learning from a very small amount of original text.

I think this is one of the main reasons the model learned local patterns such as names, common letters, and punctuation without learning how to generate a long meaningful conversation.

D. Loss Does Not Mean Perfect Text

My training loss decreased from 3.1064 to 1.8965.

That is a good sign because it means the model improved on its training objective.

At the same time, the generated text still contained many mistakes.

This was useful for me because it showed that a lower training loss does not automatically mean the model will generate fluent text.

The homework did not ask for a separate six-example failure gallery, so I used the actual failed generation patterns directly in this analysis.

IX. SAVED RESULTS AND REPRODUCIBILITY

I kept the outputs from my final experiment instead of reporting numbers from different test runs.

My homework folder contains the executed notebook together with the main supporting files such as `RUN_LOG.txt`, `METRICS.md`, `training_loss.png`, `results.json`, and the saved model checkpoint.

The model checkpoint stores the trained weights and the information needed to rebuild the same model, including the character mappings and model configuration.

The complete generated outputs are also saved in my executed notebook and run log.

The required `AI_USE.md` is kept separately in the repository as required by the course instructions.

X. CONCLUSION

This homework helped me understand the basic flow of a GPT model much better.

I started with normal text and converted every character into an integer token. I then converted those tokens into embeddings and added position information.

After that, I manually implemented multi-head attention and used a causal mask so that the model could not see future characters. I combined attention with LayerNorm, residual connections, and a GELU feedforward network to build the decoder blocks.

The final model used four attention heads, two decoder blocks, and 425,777 trainable parameters.

After eight epochs, the training loss decreased from 3.1064 to 1.8965, showing that the model was learning patterns from the provided text.

The generation part was especially useful because the decoding methods behaved differently even though they all used the exact same trained model.

Greedy decoding became repetitive. Lower temperature was more controlled. Higher temperature produced more variation. Top- k with a small k limited the choices and gave somewhat more readable fragments.

For my final run, top- k with $k = 2$ was the most readable among the tested outputs, while temperature 1.5 produced the most diversity.

The generated text was not perfect, but that was also an important part of the experiment. The dataset was very small,

so I did not expect this Mini GPT to behave like a large pretrained language model.

Overall, building each part manually helped me connect the concepts of tokenization, embeddings, attention, masking, training, and text generation instead of treating GPT as a single ready-made model.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [3] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.